

week 2: linear model and causal inference

geocentric models

Announcements and such

- We will start using brms today! Install this package now, if you haven't already.

```
install.packages(c("brms", "tidybayes"))
```

- No office hours Friday
- [Lori Olsen](#)
- Projects

Workspace setup

```
library(here)
library(tidyverse)
library(cowplot)
library(brms)
library(tidybayes)
library(patchwork)
```

How to draw an owl

1.



2.



1. Draw some circles

2. Draw the rest of the fucking owl

Workflow

1. State a clear question.
2. Sketch your causal assumptions.
3. Use the sketch to define a generative model.
4. Use the generative model to build an estimator.
5. Profit.

Model “recipes”

1. Recognize a set of variables to work with. (Data and parameters.)
2. Define each variable either in terms of the other variables OR in terms of a probability distribution.
3. The combination of variables and their probability distributions defines a **joint generative model** that can be used to simulate hypothetical observations and analyze real ones.

Here’s an example:

$$\begin{aligned}y_i &\sim \text{Normal}(\mu_i, \sigma) \\ \mu_i &= \beta x_i \\ \beta &\sim \text{Normal}(0, 10) \\ \sigma &\sim \text{Exponential}(1) \\ x_i &\sim \text{Normal}(0, 1)\end{aligned}$$

Model for globe-tossing

Here’s the model for last week’s globe-tossing experiment:

$$\begin{aligned}W &\sim \text{Binomial}(N, p) \\ p &\sim \text{Uniform}(0, 1)\end{aligned}$$

- W is the observed count of water.
- N is the total number of tosses.
- p is the proportion of water on the globe.

The whole model can be read as:

The count W is distributed binomially with sample size N and probability p . The prior for p is assumed to be uniform between 0 and 1.

Model for globe-tossing

Here's the model for last week's globe-tossing experiment:

$$\begin{aligned}W &\sim \text{Binomial}(N, p) \\ p &\sim \text{Uniform}(0, 1)\end{aligned}$$

Estimating the posterior using brms

Last week, we used grid approximation to estimate the posterior distribution. In the video you watched for today, McElreath moves on to something called **QUADRATIC APPROXIMATION**. It's good to understand what that's doing, but you and I are moving right along to MCMC. We won't go into the details of what's happening for a few weeks, but let's start with the code to estimate the model.

Let's say we tossed the globe 9 times and observed 6 waters:

```
m1 <-  
  brm(data = list(w = 6),  
       family = binomial(link = "identity"),  
       w | trials(9) ~ 0 + Intercept,  
       prior(uniform(0, 1), class = b),  
       iter = 5000, warmup = 1000, seed = 3, chains=1,  
       file = here("files/models/m21.1"))
```

- ① Data can be a data frame or a list. Just make sure the variable names match what's in your formula.
- ② How you assume your **outcome** variable is distributed.
- ③ The formula for your outcome.
- ④ Priors for *every* parameter in your model. Here, we only have one parameter, so we only need 1 prior. This happens to be a flat prior between 0 and 1.
- ⑤ Some choices about how we want our model to run. We'll go more into this later.
- ⑥ These models can take a long time to run. You can automatically save the output to a file; when you do this, the next time you run this code, it won't actually estimate the model, but will instead pull the output from your stated file. Be WARNED: if you change the data or the model code, it will NOT reestimate your model until you delete the file.

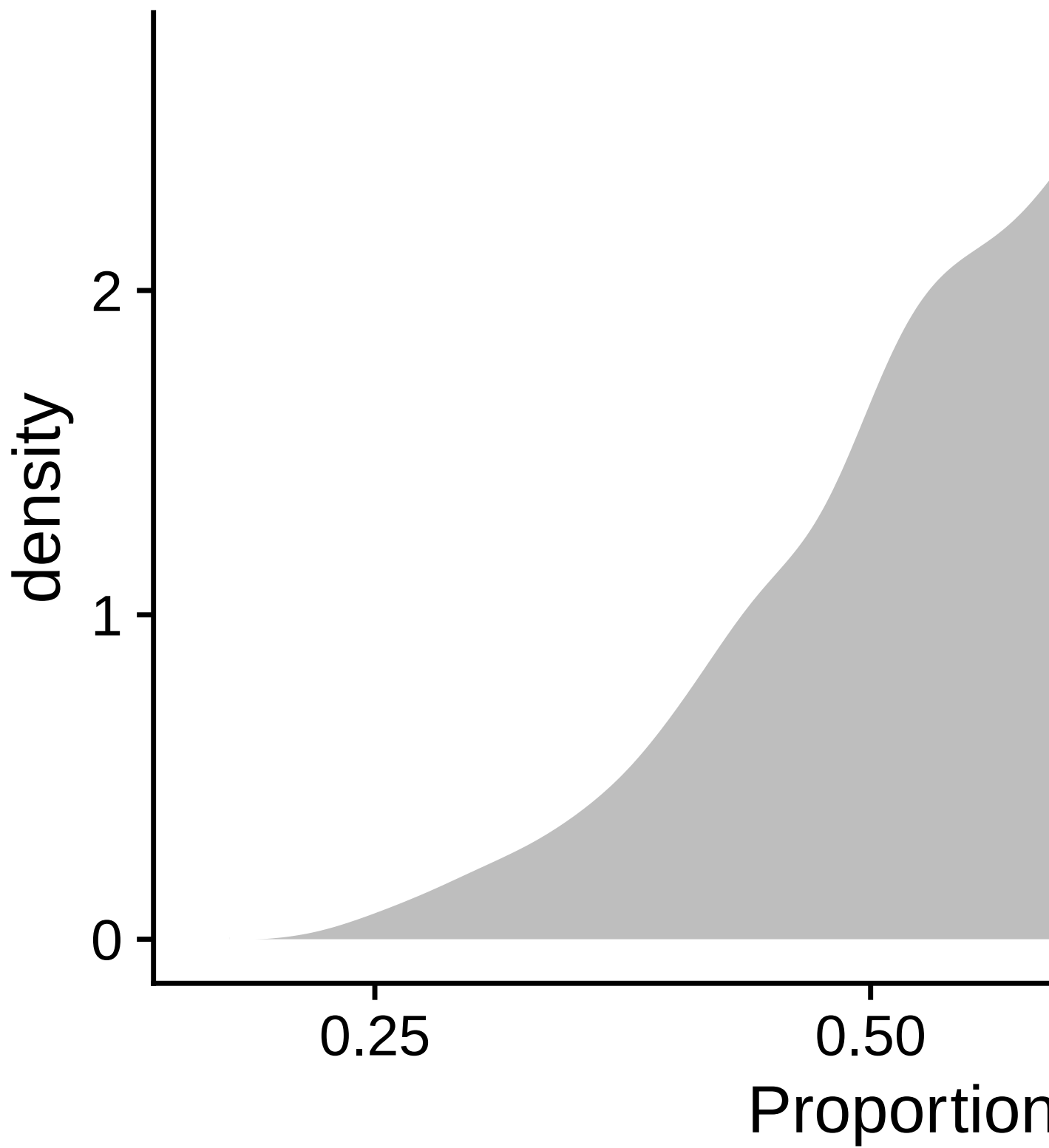
Sampling from the posterior

Grid approximation gave us the calculated probability of each possible value of our parameter, p . But our method of conducting bayes will no longer give us such a neat solution. Here's how you get the posterior distribution for p :

```
samples_from_post = as_draws_df(m1)
samples_from_post
```

```
# A draws_df: 4000 iterations, 1 chains, and 3 variables
  b_Intercept lprior lp__
1      0.46      0 -2.1
2      0.49      0 -1.9
3      0.56      0 -1.5
4      0.66      0 -1.3
5      0.71      0 -1.3
6      0.62      0 -1.3
7      0.68      0 -1.3
8      0.55      0 -1.5
9      0.63      0 -1.3
10     0.67      0 -1.3
# ... with 3990 more draws
# ... hidden reserved variables {'.chain', '.iteration', '.draw'}
```

```
samples_from_post %>%
  ggplot(aes(x=b_Intercept)) +
  geom_density(fill = "grey", color = "white") +
  labs(x="Proportion water")
```



Posterior predictive distribution

```
ppd = posterior_predict(m1)
dim(ppd)
```

```
[1] 4000    1
```

```
ppd
```

```
      [,1]
 [1,]    6
 [2,]    3
 [3,]    6
 [4,]    5
 [5,]    6
 [6,]    5
 [7,]    5
 [8,]    6
 [9,]    8
[10,]    8
[11,]    5
[12,]    4
[13,]    5
[14,]    9
[15,]    4
[16,]    8
[17,]    8
[18,]    7
[19,]    6
[20,]    6
[21,]    9
[22,]    9
[23,]    6
[24,]    5
[25,]    5
[26,]    4
[27,]    9
[28,]    6
```

[29,]	6
[30,]	4
[31,]	4
[32,]	7
[33,]	6
[34,]	5
[35,]	2
[36,]	4
[37,]	2
[38,]	5
[39,]	3
[40,]	6
[41,]	6
[42,]	8
[43,]	8
[44,]	6
[45,]	6
[46,]	5
[47,]	9
[48,]	5
[49,]	5
[50,]	7
[51,]	5
[52,]	5
[53,]	7
[54,]	4
[55,]	3
[56,]	3
[57,]	5
[58,]	3
[59,]	5
[60,]	3
[61,]	6
[62,]	6
[63,]	7
[64,]	4
[65,]	7
[66,]	4
[67,]	5
[68,]	4
[69,]	4
[70,]	8
[71,]	8

[72,]	1
[73,]	6
[74,]	7
[75,]	5
[76,]	5
[77,]	5
[78,]	7
[79,]	9
[80,]	6
[81,]	6
[82,]	8
[83,]	5
[84,]	9
[85,]	7
[86,]	8
[87,]	7
[88,]	5
[89,]	5
[90,]	8
[91,]	6
[92,]	6
[93,]	6
[94,]	7
[95,]	4
[96,]	7
[97,]	8
[98,]	8
[99,]	8
[100,]	7
[101,]	5
[102,]	8
[103,]	5
[104,]	6
[105,]	6
[106,]	4
[107,]	4
[108,]	8
[109,]	6
[110,]	6
[111,]	5
[112,]	4
[113,]	3
[114,]	7

[115,]	6
[116,]	5
[117,]	6
[118,]	3
[119,]	6
[120,]	3
[121,]	8
[122,]	2
[123,]	6
[124,]	7
[125,]	8
[126,]	7
[127,]	7
[128,]	5
[129,]	5
[130,]	8
[131,]	7
[132,]	9
[133,]	6
[134,]	6
[135,]	8
[136,]	8
[137,]	4
[138,]	5
[139,]	8
[140,]	8
[141,]	5
[142,]	6
[143,]	8
[144,]	6
[145,]	6
[146,]	9
[147,]	8
[148,]	5
[149,]	5
[150,]	5
[151,]	4
[152,]	8
[153,]	8
[154,]	5
[155,]	5
[156,]	8
[157,]	8

[158,]	5
[159,]	7
[160,]	4
[161,]	6
[162,]	4
[163,]	7
[164,]	7
[165,]	6
[166,]	3
[167,]	6
[168,]	6
[169,]	7
[170,]	3
[171,]	6
[172,]	1
[173,]	6
[174,]	6
[175,]	5
[176,]	5
[177,]	8
[178,]	9
[179,]	8
[180,]	8
[181,]	7
[182,]	8
[183,]	6
[184,]	4
[185,]	5
[186,]	8
[187,]	4
[188,]	8
[189,]	3
[190,]	3
[191,]	7
[192,]	6
[193,]	9
[194,]	7
[195,]	5
[196,]	4
[197,]	6
[198,]	6
[199,]	6
[200,]	6

[201,]	8
[202,]	7
[203,]	9
[204,]	8
[205,]	6
[206,]	7
[207,]	8
[208,]	6
[209,]	8
[210,]	7
[211,]	8
[212,]	7
[213,]	7
[214,]	9
[215,]	8
[216,]	8
[217,]	7
[218,]	5
[219,]	3
[220,]	2
[221,]	7
[222,]	6
[223,]	5
[224,]	7
[225,]	6
[226,]	6
[227,]	5
[228,]	6
[229,]	9
[230,]	8
[231,]	8
[232,]	4
[233,]	6
[234,]	8
[235,]	5
[236,]	7
[237,]	7
[238,]	5
[239,]	6
[240,]	4
[241,]	3
[242,]	5
[243,]	7

[244,]	8
[245,]	5
[246,]	5
[247,]	5
[248,]	5
[249,]	7
[250,]	6
[251,]	7
[252,]	5
[253,]	3
[254,]	6
[255,]	9
[256,]	8
[257,]	6
[258,]	4
[259,]	4
[260,]	8
[261,]	6
[262,]	6
[263,]	4
[264,]	9
[265,]	8
[266,]	8
[267,]	3
[268,]	4
[269,]	6
[270,]	6
[271,]	6
[272,]	6
[273,]	6
[274,]	7
[275,]	4
[276,]	9
[277,]	5
[278,]	7
[279,]	8
[280,]	9
[281,]	5
[282,]	6
[283,]	3
[284,]	4
[285,]	3
[286,]	4

[287,]	2
[288,]	9
[289,]	6
[290,]	8
[291,]	7
[292,]	8
[293,]	8
[294,]	6
[295,]	3
[296,]	5
[297,]	8
[298,]	4
[299,]	2
[300,]	3
[301,]	5
[302,]	8
[303,]	3
[304,]	7
[305,]	8
[306,]	7
[307,]	7
[308,]	7
[309,]	6
[310,]	7
[311,]	4
[312,]	5
[313,]	6
[314,]	4
[315,]	5
[316,]	7
[317,]	6
[318,]	7
[319,]	7
[320,]	7
[321,]	7
[322,]	4
[323,]	3
[324,]	4
[325,]	5
[326,]	3
[327,]	5
[328,]	8
[329,]	7

[330,]	4
[331,]	5
[332,]	5
[333,]	9
[334,]	7
[335,]	8
[336,]	8
[337,]	8
[338,]	4
[339,]	3
[340,]	2
[341,]	5
[342,]	4
[343,]	4
[344,]	8
[345,]	8
[346,]	6
[347,]	6
[348,]	6
[349,]	8
[350,]	6
[351,]	9
[352,]	6
[353,]	6
[354,]	6
[355,]	7
[356,]	4
[357,]	6
[358,]	5
[359,]	3
[360,]	5
[361,]	5
[362,]	7
[363,]	4
[364,]	4
[365,]	6
[366,]	5
[367,]	5
[368,]	4
[369,]	8
[370,]	8
[371,]	3
[372,]	5

[373,]	4
[374,]	3
[375,]	3
[376,]	3
[377,]	3
[378,]	5
[379,]	4
[380,]	4
[381,]	4
[382,]	4
[383,]	6
[384,]	6
[385,]	8
[386,]	8
[387,]	6
[388,]	8
[389,]	2
[390,]	6
[391,]	5
[392,]	4
[393,]	5
[394,]	7
[395,]	7
[396,]	8
[397,]	8
[398,]	7
[399,]	6
[400,]	6
[401,]	5
[402,]	6
[403,]	8
[404,]	4
[405,]	5
[406,]	8
[407,]	8
[408,]	9
[409,]	9
[410,]	5
[411,]	7
[412,]	7
[413,]	6
[414,]	4
[415,]	5

[416,]	7
[417,]	5
[418,]	5
[419,]	4
[420,]	5
[421,]	2
[422,]	4
[423,]	4
[424,]	6
[425,]	8
[426,]	4
[427,]	1
[428,]	8
[429,]	7
[430,]	8
[431,]	5
[432,]	3
[433,]	3
[434,]	8
[435,]	6
[436,]	8
[437,]	8
[438,]	4
[439,]	8
[440,]	7
[441,]	7
[442,]	7
[443,]	5
[444,]	9
[445,]	4
[446,]	3
[447,]	6
[448,]	3
[449,]	8
[450,]	5
[451,]	5
[452,]	4
[453,]	7
[454,]	9
[455,]	7
[456,]	7
[457,]	5
[458,]	7

[459,]	6
[460,]	6
[461,]	6
[462,]	8
[463,]	9
[464,]	7
[465,]	8
[466,]	3
[467,]	7
[468,]	4
[469,]	8
[470,]	4
[471,]	4
[472,]	6
[473,]	7
[474,]	7
[475,]	4
[476,]	8
[477,]	6
[478,]	4
[479,]	7
[480,]	8
[481,]	8
[482,]	3
[483,]	5
[484,]	6
[485,]	3
[486,]	5
[487,]	8
[488,]	2
[489,]	6
[490,]	3
[491,]	9
[492,]	7
[493,]	5
[494,]	7
[495,]	6
[496,]	6
[497,]	6
[498,]	6
[499,]	6
[500,]	5
[501,]	4

[502,]	6
[503,]	6
[504,]	5
[505,]	6
[506,]	5
[507,]	8
[508,]	6
[509,]	6
[510,]	8
[511,]	8
[512,]	9
[513,]	7
[514,]	6
[515,]	6
[516,]	4
[517,]	4
[518,]	5
[519,]	7
[520,]	5
[521,]	2
[522,]	3
[523,]	3
[524,]	5
[525,]	8
[526,]	8
[527,]	8
[528,]	4
[529,]	5
[530,]	5
[531,]	6
[532,]	7
[533,]	7
[534,]	7
[535,]	7
[536,]	7
[537,]	7
[538,]	7
[539,]	6
[540,]	6
[541,]	3
[542,]	7
[543,]	6
[544,]	6

[545,]	7
[546,]	6
[547,]	5
[548,]	6
[549,]	6
[550,]	4
[551,]	9
[552,]	8
[553,]	6
[554,]	7
[555,]	5
[556,]	4
[557,]	0
[558,]	2
[559,]	2
[560,]	8
[561,]	6
[562,]	7
[563,]	4
[564,]	4
[565,]	7
[566,]	4
[567,]	3
[568,]	7
[569,]	7
[570,]	5
[571,]	7
[572,]	8
[573,]	2
[574,]	3
[575,]	4
[576,]	9
[577,]	6
[578,]	7
[579,]	6
[580,]	7
[581,]	8
[582,]	6
[583,]	7
[584,]	7
[585,]	4
[586,]	3
[587,]	1

[588,]	3
[589,]	2
[590,]	0
[591,]	5
[592,]	4
[593,]	4
[594,]	7
[595,]	4
[596,]	6
[597,]	2
[598,]	4
[599,]	8
[600,]	5
[601,]	5
[602,]	3
[603,]	2
[604,]	9
[605,]	8
[606,]	7
[607,]	9
[608,]	4
[609,]	7
[610,]	6
[611,]	5
[612,]	7
[613,]	7
[614,]	4
[615,]	5
[616,]	2
[617,]	6
[618,]	7
[619,]	6
[620,]	5
[621,]	6
[622,]	5
[623,]	7
[624,]	6
[625,]	6
[626,]	6
[627,]	6
[628,]	7
[629,]	5
[630,]	5

[631,]	3
[632,]	6
[633,]	5
[634,]	2
[635,]	6
[636,]	7
[637,]	2
[638,]	4
[639,]	3
[640,]	2
[641,]	4
[642,]	6
[643,]	8
[644,]	2
[645,]	3
[646,]	5
[647,]	5
[648,]	4
[649,]	5
[650,]	6
[651,]	6
[652,]	7
[653,]	7
[654,]	8
[655,]	7
[656,]	9
[657,]	8
[658,]	5
[659,]	8
[660,]	5
[661,]	8
[662,]	3
[663,]	2
[664,]	3
[665,]	3
[666,]	6
[667,]	9
[668,]	7
[669,]	5
[670,]	5
[671,]	7
[672,]	4
[673,]	6

[674,]	6
[675,]	7
[676,]	4
[677,]	7
[678,]	3
[679,]	9
[680,]	4
[681,]	5
[682,]	4
[683,]	5
[684,]	5
[685,]	6
[686,]	7
[687,]	5
[688,]	8
[689,]	8
[690,]	7
[691,]	7
[692,]	8
[693,]	5
[694,]	7
[695,]	6
[696,]	8
[697,]	4
[698,]	4
[699,]	2
[700,]	4
[701,]	4
[702,]	7
[703,]	4
[704,]	3
[705,]	3
[706,]	7
[707,]	4
[708,]	4
[709,]	9
[710,]	7
[711,]	4
[712,]	6
[713,]	7
[714,]	7
[715,]	3
[716,]	8

[717,]	8
[718,]	5
[719,]	3
[720,]	5
[721,]	3
[722,]	2
[723,]	4
[724,]	8
[725,]	7
[726,]	5
[727,]	5
[728,]	5
[729,]	5
[730,]	3
[731,]	5
[732,]	6
[733,]	6
[734,]	9
[735,]	6
[736,]	9
[737,]	5
[738,]	6
[739,]	6
[740,]	7
[741,]	7
[742,]	7
[743,]	6
[744,]	3
[745,]	7
[746,]	9
[747,]	4
[748,]	5
[749,]	6
[750,]	8
[751,]	5
[752,]	5
[753,]	5
[754,]	5
[755,]	5
[756,]	5
[757,]	6
[758,]	7
[759,]	5

[760,]	6
[761,]	5
[762,]	7
[763,]	6
[764,]	6
[765,]	6
[766,]	9
[767,]	7
[768,]	7
[769,]	8
[770,]	4
[771,]	6
[772,]	5
[773,]	3
[774,]	6
[775,]	8
[776,]	7
[777,]	4
[778,]	9
[779,]	4
[780,]	5
[781,]	5
[782,]	6
[783,]	4
[784,]	2
[785,]	6
[786,]	3
[787,]	0
[788,]	6
[789,]	6
[790,]	9
[791,]	6
[792,]	8
[793,]	8
[794,]	5
[795,]	8
[796,]	4
[797,]	9
[798,]	8
[799,]	9
[800,]	6
[801,]	9
[802,]	8

[803,]	9
[804,]	8
[805,]	8
[806,]	7
[807,]	6
[808,]	6
[809,]	6
[810,]	8
[811,]	6
[812,]	4
[813,]	7
[814,]	8
[815,]	8
[816,]	6
[817,]	8
[818,]	6
[819,]	6
[820,]	7
[821,]	4
[822,]	5
[823,]	6
[824,]	6
[825,]	5
[826,]	4
[827,]	2
[828,]	5
[829,]	4
[830,]	7
[831,]	6
[832,]	3
[833,]	6
[834,]	5
[835,]	4
[836,]	4
[837,]	7
[838,]	4
[839,]	6
[840,]	5
[841,]	7
[842,]	9
[843,]	9
[844,]	7
[845,]	8

[846,]	5
[847,]	5
[848,]	4
[849,]	3
[850,]	6
[851,]	5
[852,]	8
[853,]	6
[854,]	7
[855,]	8
[856,]	4
[857,]	6
[858,]	3
[859,]	4
[860,]	7
[861,]	6
[862,]	4
[863,]	8
[864,]	5
[865,]	7
[866,]	8
[867,]	8
[868,]	7
[869,]	9
[870,]	6
[871,]	7
[872,]	4
[873,]	7
[874,]	6
[875,]	4
[876,]	3
[877,]	1
[878,]	4
[879,]	1
[880,]	6
[881,]	2
[882,]	9
[883,]	9
[884,]	7
[885,]	5
[886,]	7
[887,]	7
[888,]	7

[889,]	3
[890,]	8
[891,]	7
[892,]	7
[893,]	3
[894,]	5
[895,]	7
[896,]	4
[897,]	7
[898,]	7
[899,]	5
[900,]	5
[901,]	6
[902,]	7
[903,]	4
[904,]	6
[905,]	8
[906,]	9
[907,]	8
[908,]	6
[909,]	8
[910,]	8
[911,]	9
[912,]	8
[913,]	8
[914,]	8
[915,]	6
[916,]	6
[917,]	6
[918,]	3
[919,]	7
[920,]	3
[921,]	5
[922,]	4
[923,]	4
[924,]	5
[925,]	4
[926,]	5
[927,]	2
[928,]	4
[929,]	5
[930,]	8
[931,]	7

[932,]	8
[933,]	6
[934,]	7
[935,]	3
[936,]	3
[937,]	8
[938,]	6
[939,]	7
[940,]	8
[941,]	6
[942,]	4
[943,]	6
[944,]	8
[945,]	5
[946,]	5
[947,]	5
[948,]	4
[949,]	7
[950,]	3
[951,]	7
[952,]	5
[953,]	5
[954,]	4
[955,]	3
[956,]	5
[957,]	4
[958,]	3
[959,]	8
[960,]	8
[961,]	7
[962,]	9
[963,]	7
[964,]	2
[965,]	5
[966,]	2
[967,]	3
[968,]	7
[969,]	5
[970,]	3
[971,]	4
[972,]	7
[973,]	3
[974,]	5

```

[975,] 4
[976,] 6
[977,] 3
[978,] 9
[979,] 7
[980,] 7
[981,] 6
[982,] 7
[983,] 4
[984,] 9
[985,] 5
[986,] 6
[987,] 2
[988,] 8
[989,] 2
[990,] 5
[991,] 2
[992,] 8
[993,] 8
[994,] 8
[995,] 6
[996,] 8
[997,] 3
[998,] 4
[999,] 6
[1000,] 4
[ reached 'max' / getOption("max.print") -- omitted 3000 rows ]

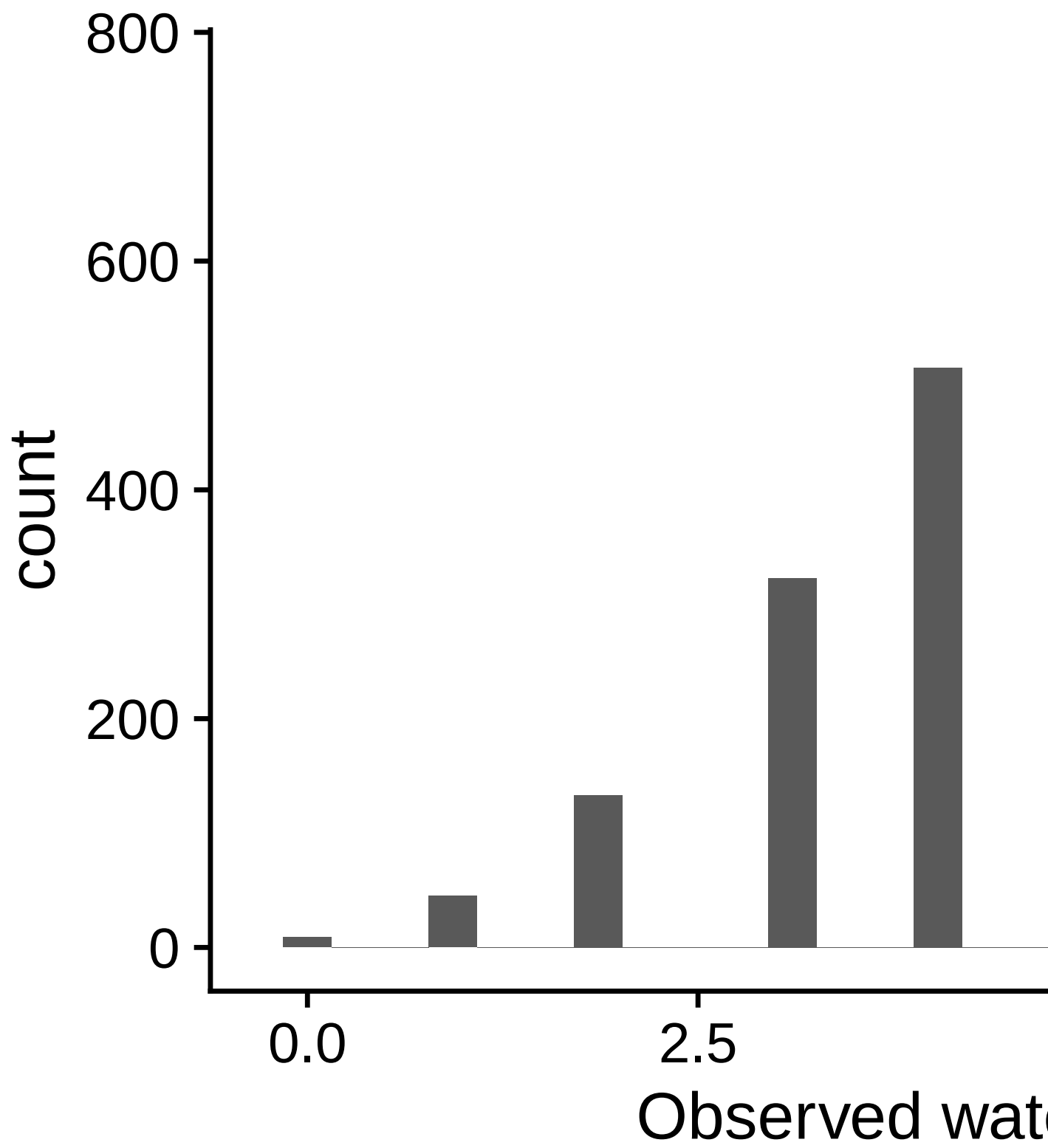
```

```

data.frame(obs = ppd) %>%
  ggplot(aes(x=obs)) +
  geom_histogram() +
  labs(x="Observed water (out of 9)")

```

``stat_bin()`` using ``bins = 30``. Pick better value ``binwidth``.



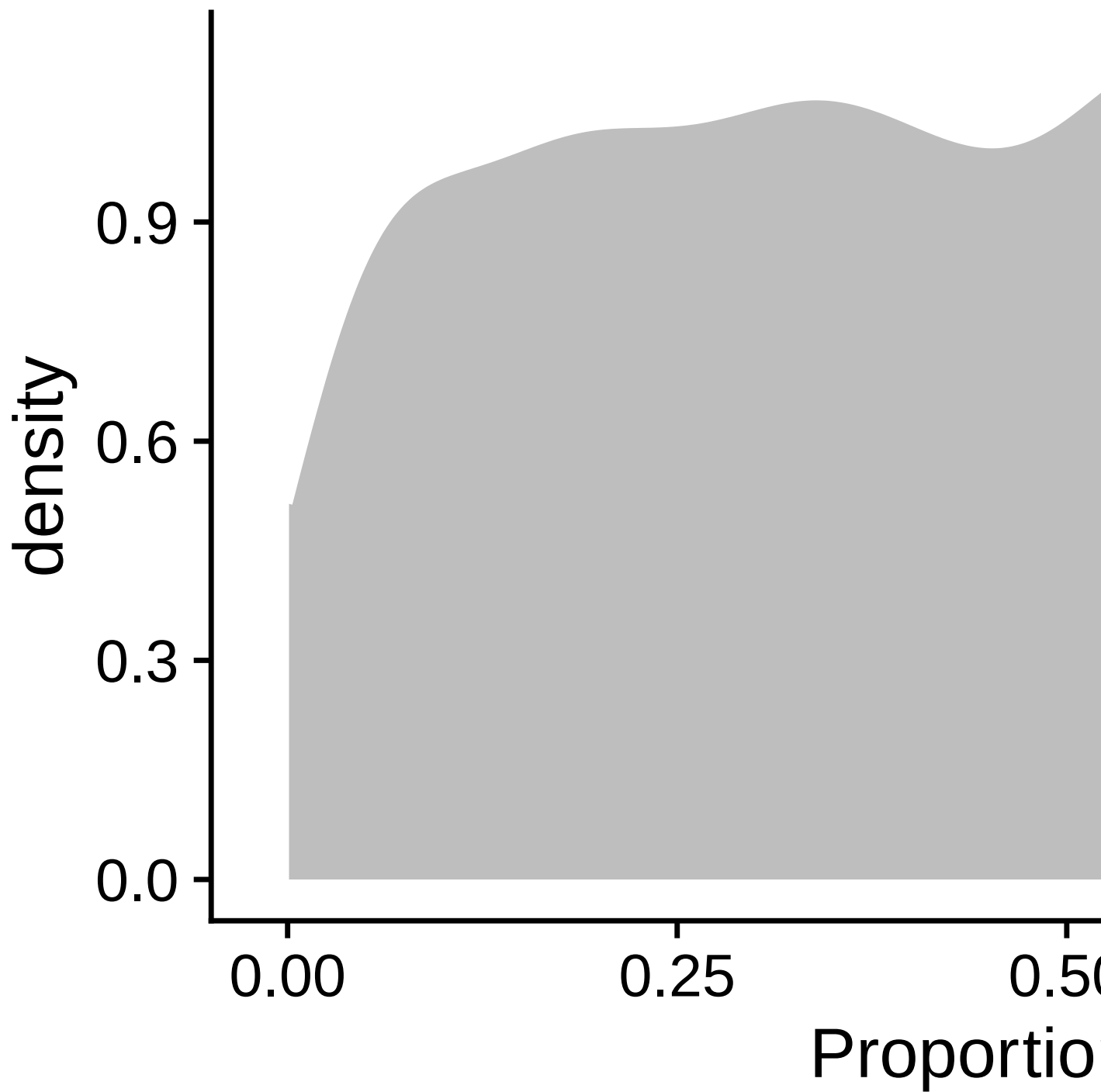
Sampling parameters from the prior

Oops, we've jumped ahead of ourselves! Best practice is to simulate values from your prior **first** and check to see if those priors are reasonable.

```
1 m1p <-  
2   brm(data = list(w = 6),  
3       family = binomial(link = "identity"),  
4       w | trials(9) ~ 0 + Intercept,  
5       prior(beta(1, 1), class = b, lb = 0, ub = 1),  
6       iter = 5000, warmup = 1000, seed = 3, chains=1,  
7       sample_prior = "only")
```

```
samples_from_prior = as_draws_df(m1p)  
samples_from_prior %>%  
  ggplot(aes(x=b_Intercept)) +  
  geom_density(fill = "grey", color = "white") +  
  labs(x="Proportion water", title="Prior")
```

Prior

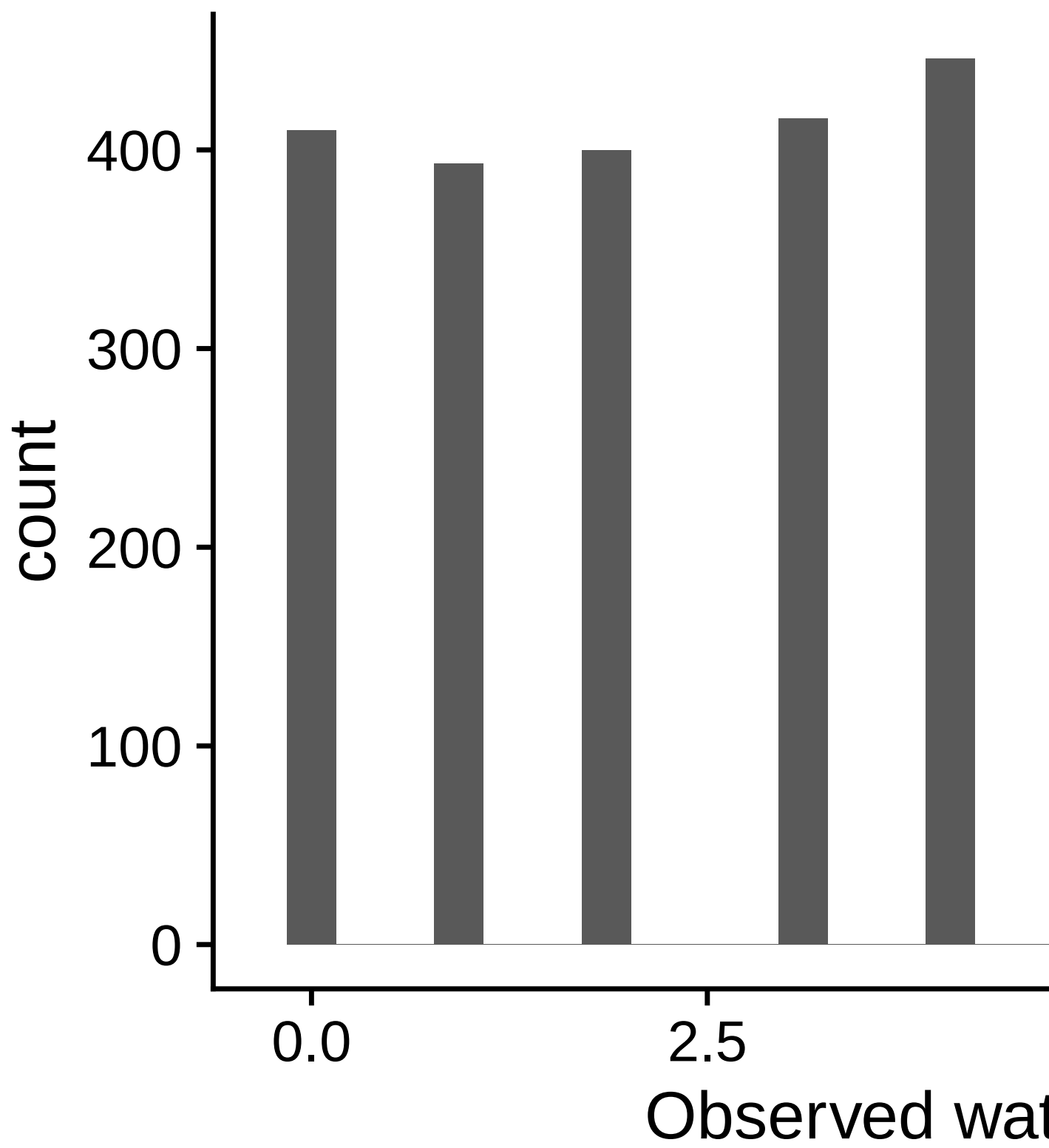


Simulating observations from the prior

We may also want the **PRIOR PREDICTIVE DISTRIBUTION** which is the expected observations given our prior.

```
prior_pd = posterior_predict(m1p)
data.frame(obs = prior_pd) %>%
  ggplot(aes(x=obs)) +
  geom_histogram() +
  labs(x="Observed water (out of 9)")
```

``stat_bin()`` using ``bins = 30``. Pick better value ``binwidth``.



Simulating from your priors – **prior predictive simulation** – is an essential part of modeling. This allows you to see what your choices imply about the data. You’ll be able to diagnose bad choices.

an aside about learning in R

At this point in the course, I’m going to start throwing a lot of code at you. Do I expect you to memorize this code? Of course not.

Do you need to understand every single thing that’s happening in the code? Nope.

But, you’ll learn a lot by taking the time to figure out what’s happening in a code chunk. Class time will frequently include exercises where I ask you to adapt code I’ve shared in the slides to a new dataset or to answer a new problem. When doing so, go back through the old code and figure out what’s going on. Run the code one line at a time. Always observe the output and take some time to look at the object that was created or modified. Here are some functions that will be extremely useful:

```
str() # what kind of object is this? what is its structure?
dim() # what are the dimensions (rows/columns) of this object
head() # give me the first bit of this object
```

```
str(prior_pd)
```

```
int [1:4000, 1] 6 8 8 6 9 7 3 4 2 4 ...
- attr(*, "dimnames")=List of 2
..$ : NULL
..$ : NULL
```

```
dim(prior_pd)
```

```
[1] 4000    1
```

```
head(prior_pd)
```

	[,1]
[1,]	6
[2,]	8
[3,]	8
[4,]	6
[5,]	9
[6,]	7

Continuous outcomes

The globe tossing example is cute and easy to work with, but let's move towards the kinds of variables we more frequently work with. Let's create a model for some outcome, y that is continuous.

$$\begin{aligned}y_i &\sim \text{Normal}(\mu, \sigma) \\ \mu &\sim \text{Normal}(0, 10) \\ \sigma &\sim \text{Uniform}(0, 5)\end{aligned}$$

```
set.seed(9)
y = rnorm(n = 31, mean = 4, sd = .5)
m2 = brm(
  data = list(y=y),
  family = gaussian,
  y ~ 1,
  prior = c(prior( normal(0,10), class=Intercept),
            prior( uniform(0,5), class=sigma)),
  iter = 5000, warmup = 1000, seed = 3, chains=1,
  file = here("files/models/m21.2")
)
```

An example: weight and height

Using the Howell data (don't load the `rethinking` package because it can interfere with `brms`).

```
data("Howell1", package = "rethinking")
d <- Howell1
str(d)
```

```
'data.frame':  544 obs. of  4 variables:
 $ height: num  152 140 137 157 145 ...
 $ weight: num  47.8 36.5 31.9 53 41.3 ...
 $ age   : num  63 63 65 41 51 35 32 27 19 54 ...
 $ male  : int   1 0 0 1 0 1 0 1 0 1 ...
```

```
library(measurements)
d$height <- conv_unit(d$height, from = "cm", to = "feet")
d$weight <- conv_unit(d$weight, from = "kg", to = "lbs")
rethinking::precis(d)
```

```
Warning in seq.default(0, 1, length = length(sparks) + 1): partial argument
match of 'length' to 'length.out'
Warning in seq.default(0, 1, length = length(sparks) + 1): partial argument
match of 'length' to 'length.out'
Warning in seq.default(0, 1, length = length(sparks) + 1): partial argument
match of 'length' to 'length.out'
Warning in seq.default(0, 1, length = length(sparks) + 1): partial argument
match of 'length' to 'length.out'
```

	mean	sd	5.5%	94.5%	histogram
height	4.5362072	0.9055921	2.661042	5.4375	
weight	78.5079631	32.4502290	20.636856	120.1583	
age	29.3443934	20.7468882	1.000000	66.1350	
male	0.4724265	0.4996986	0.000000	1.0000	

```
d2 <- d[ d$age >= 18, ]
```

exercise

Write a mathematical model for the weights in this data set. (Don't worry about predicting from other variables yet.)

solution

$$\begin{aligned}w &\sim \text{Normal}(\mu, \sigma) \\ \mu &\sim \text{Normal}(130, 20) \\ \sigma &\sim \text{Uniform}(0, 25)\end{aligned}$$

$$\begin{aligned}w &\sim \text{Normal}(\mu, \sigma) \\ \mu &\sim \text{Normal}(130, 20) \\ \sigma &\sim \text{Uniform}(0, 25)\end{aligned}$$

exercise

Simulate from your priors (parameters values and prior predictive values).

solution

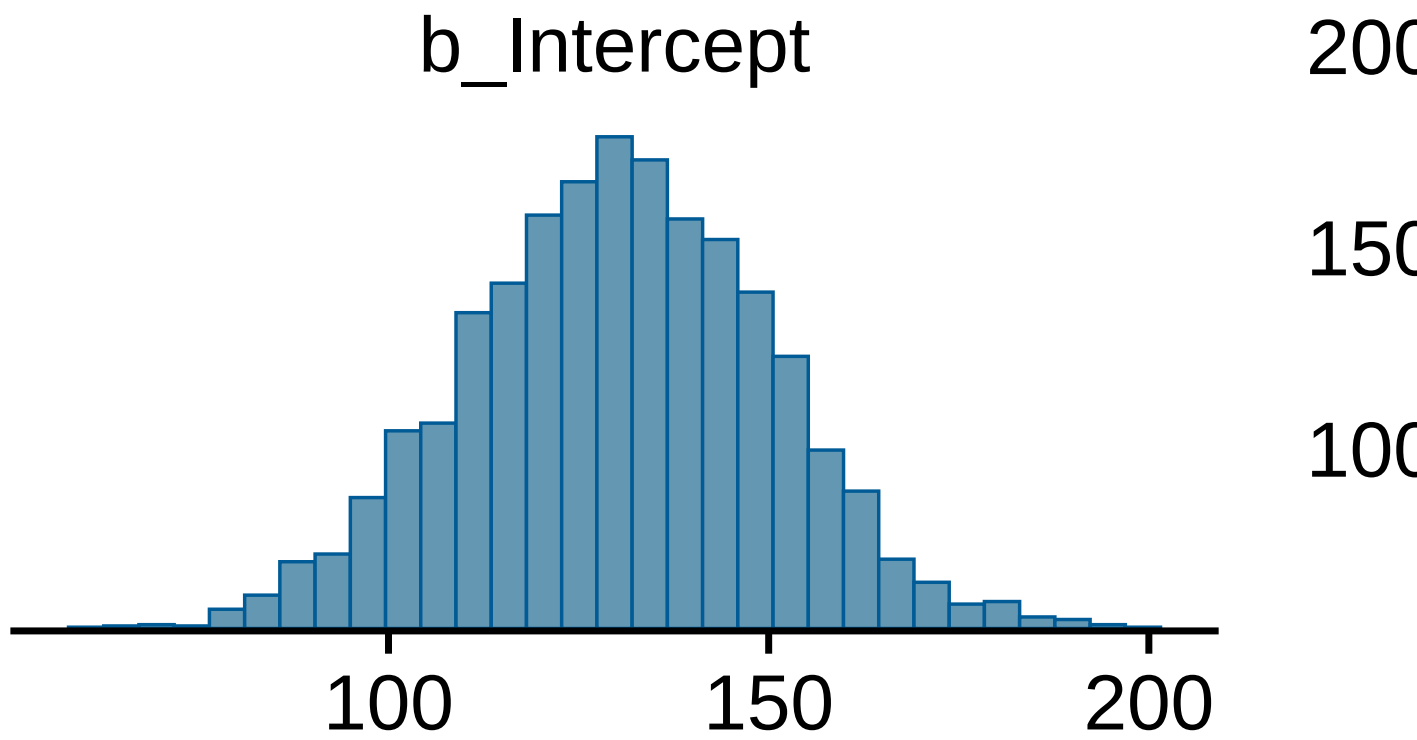
Sample from your priors:

```
m3p = brm(  
  data = d2,  
  family = gaussian,  
  weight ~ 1,  
  prior = c(prior( normal(130,20), class=Intercept),  
            prior( uniform(0,25), class=sigma, lb=0, ub=25)),  
  iter = 5000, warmup = 1000, seed = 3, chains=1,  
  sample_prior = "only")
```

Sampling parameter estimates for the Intercept.

```
pairs(m3p)
```

Warning: Only one chain in 'x'. This plot is more useful with multiple chains.



This is a different (shorter) way to plot your posterior. Good things: it automatically includes the scatterplot so you can see the implications of how these parameters correlate. Bad things: not customizable and not useable when you have a lot of parameters.

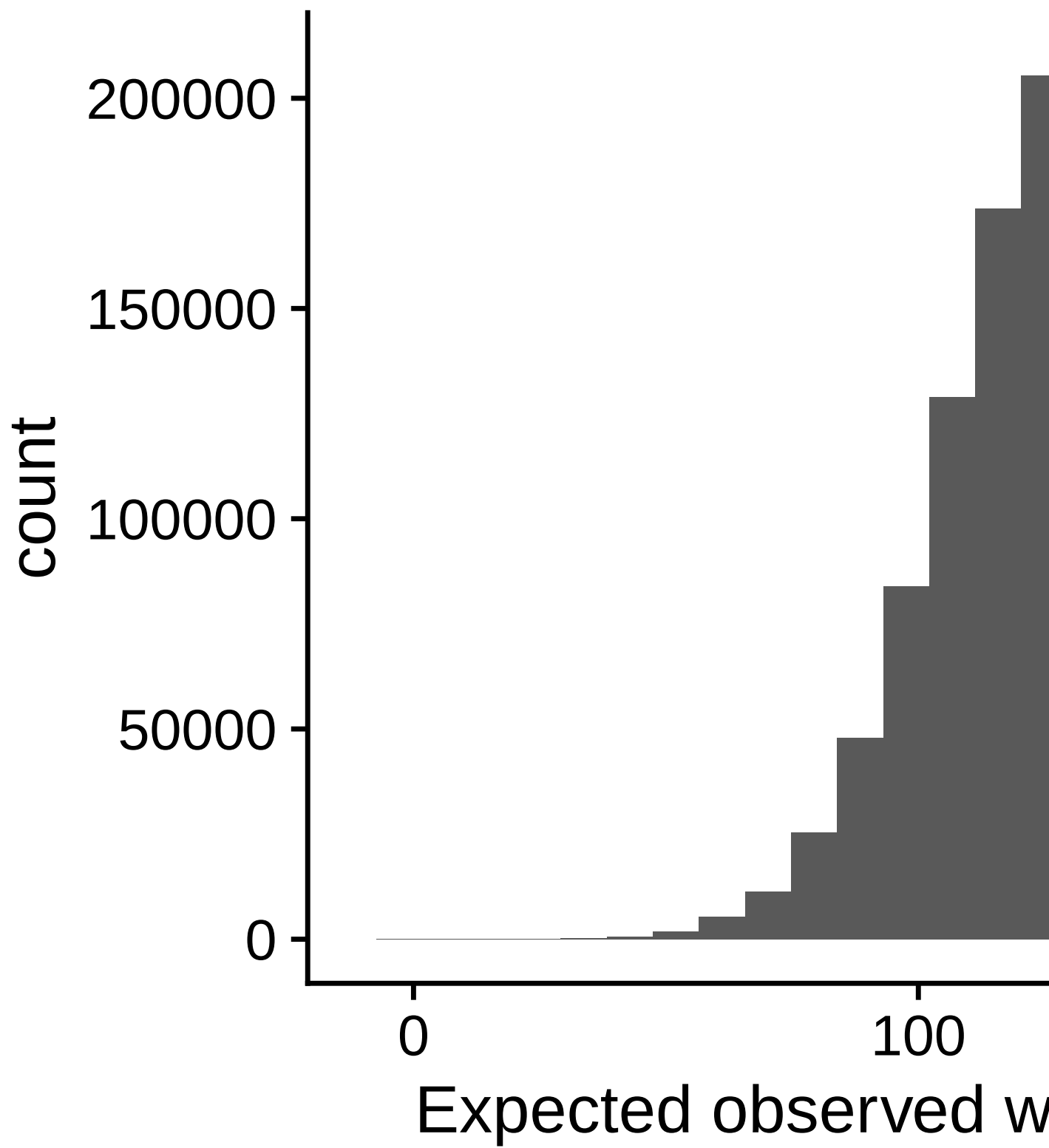
Simulate values of weight.

```
prior_pd = posterior_predict(m3p)
dim(prior_pd)
```

```
[1] 4000 352
```

```
as.data.frame(prior_pd) %>%
  pivot_longer(everything()) %>%
  ggplot(aes(x=value)) +
  geom_histogram() +
  labs(x="Expected observed weights (based on prior)")
```

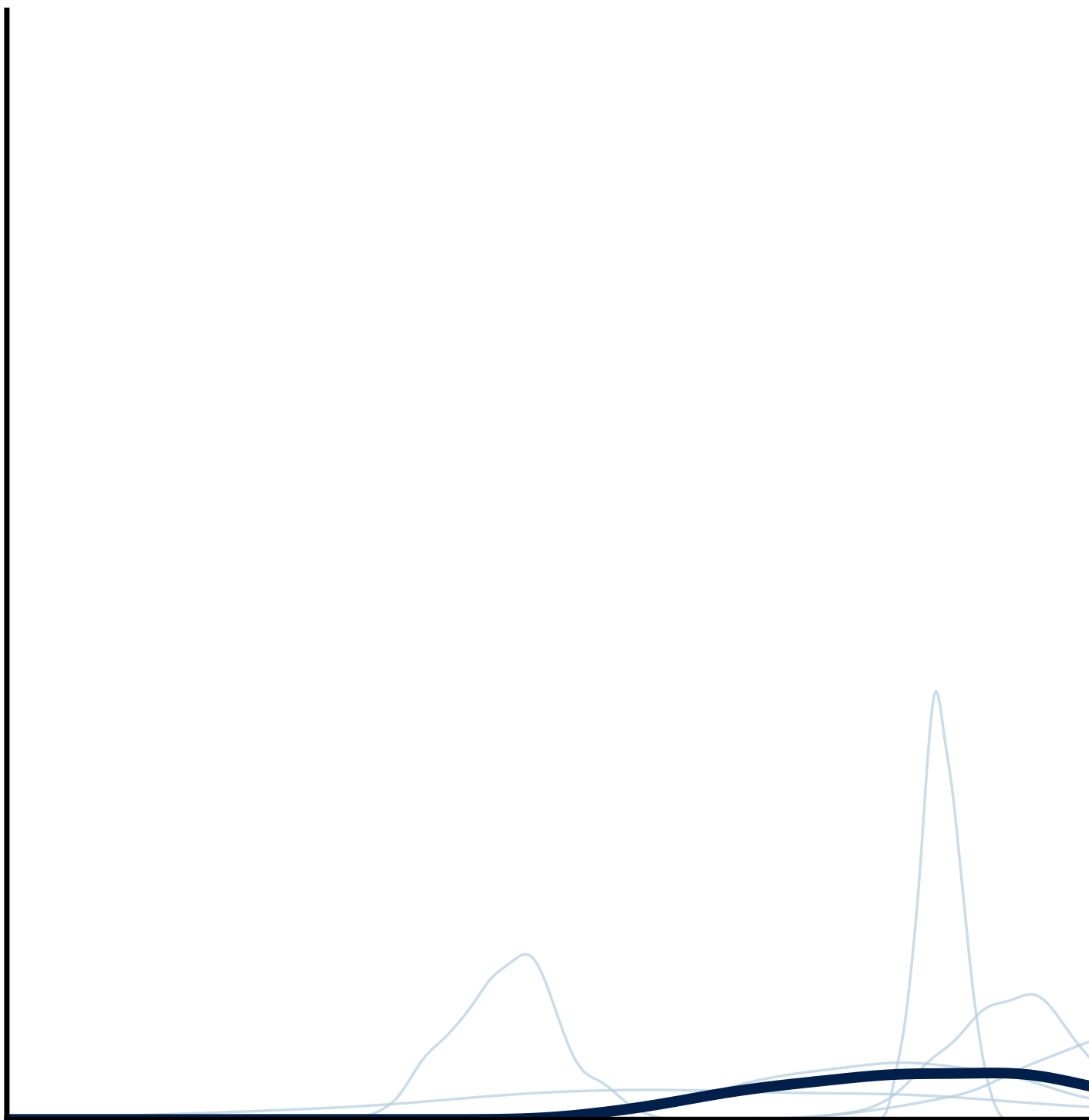
```
`stat_bin()` using `bins = 30`. Pick better value `binwidth`.
```



Another shorter way:

```
pp_check(m3p)
```

Using 10 posterior draws for ppc type 'dens_overlay' by default.



40

80

Fit the model

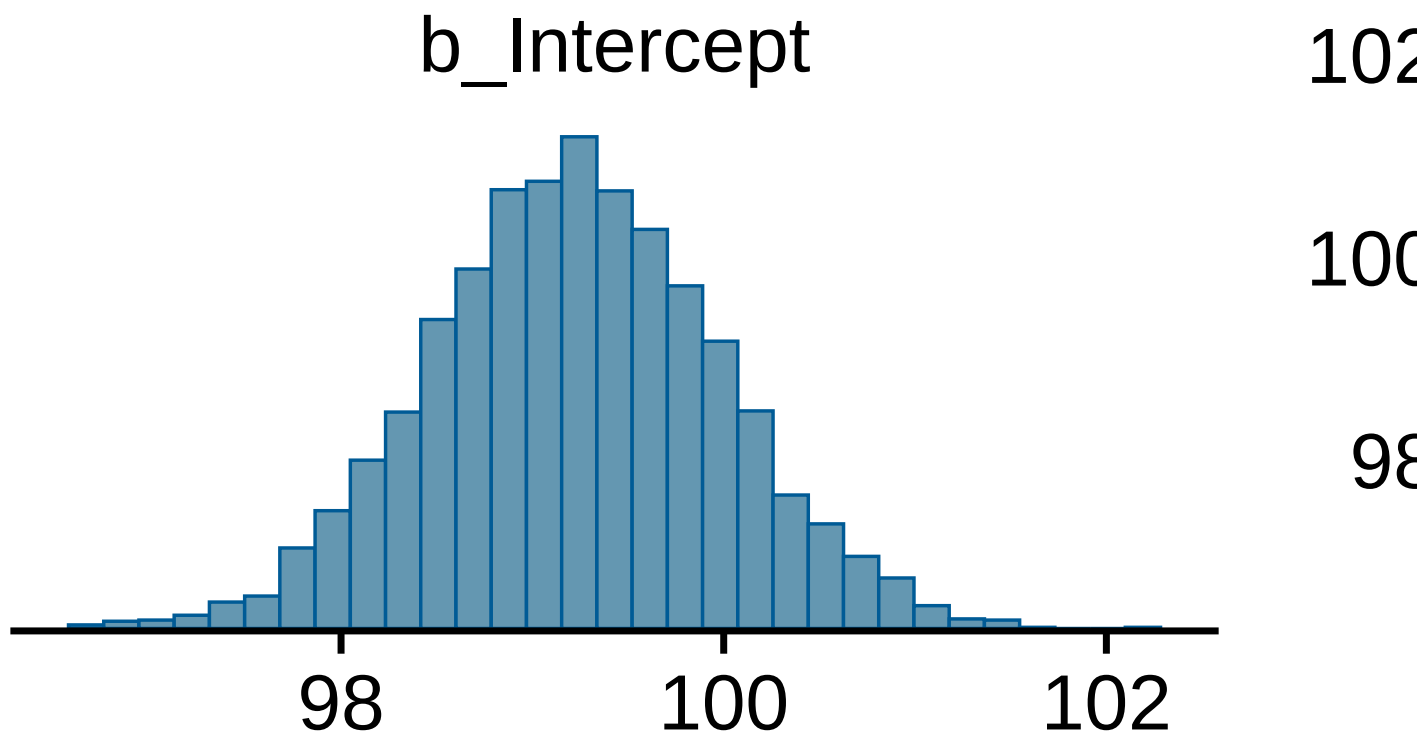
```
m3 = brm(  
  data = d2,  
  family = gaussian,  
  weight ~ 1,  
  prior = c(prior( normal(130,20), class=Intercept),  
            prior( uniform(0,25), class=sigma, lb=0, ub=25)),  
  iter = 5000, warmup = 1000, seed = 3, chains=1,  
  file = here("files/models/m21.3"))
```

```
posterior_summary(m3)
```

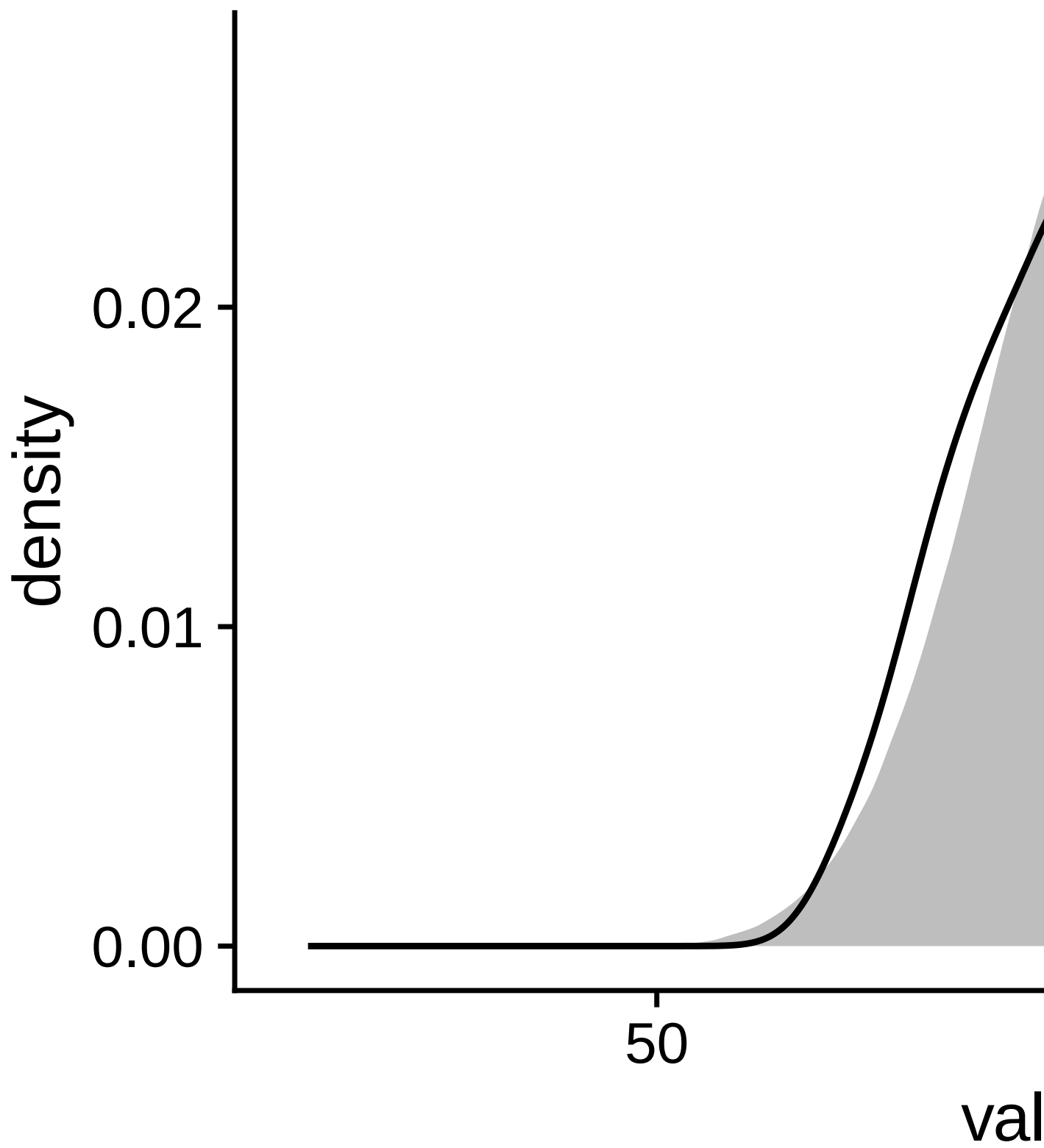
	Estimate	Est.Error	Q2.5	Q97.5
b_Intercept	99.221909	0.75720356	97.751074	100.737488
sigma	14.291987	0.55198322	13.254363	15.428183
Intercept	99.221909	0.75720356	97.751074	100.737488
lprior	-8.318377	0.05827127	-8.433538	-8.203915
lp__	-1441.294276	1.02695551	-1444.235605	-1440.292326

```
pairs(m3)
```

Warning: Only one chain in 'x'. This plot is more useful with multiple chains.

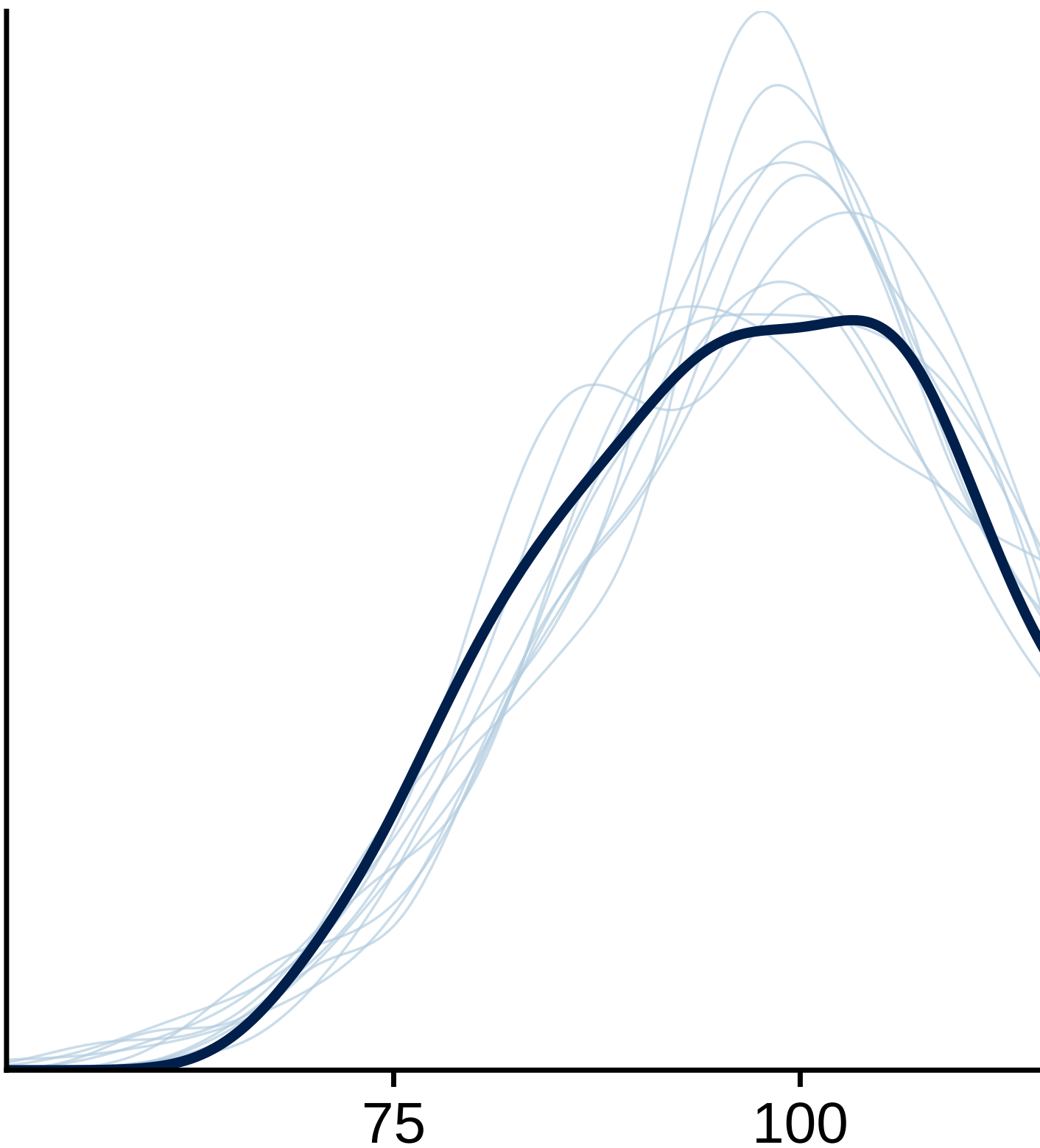


```
posterior_predict(m3) %>%  
  as.data.frame() %>%  
  pivot_longer(everything()) %>%  
  ggplot(aes(x=value)) +  
  geom_density(fill = "grey", color = "white") +  
  geom_density(aes(x = weight), data=d2, inherit.aes = F)
```



```
pp_check(m3)
```

Using 10 posterior draws for ppc type 'dens_overlay' by default.



Adding in a linear component

We might assume that height and weight are associated with each other. Indeed, within our sample:

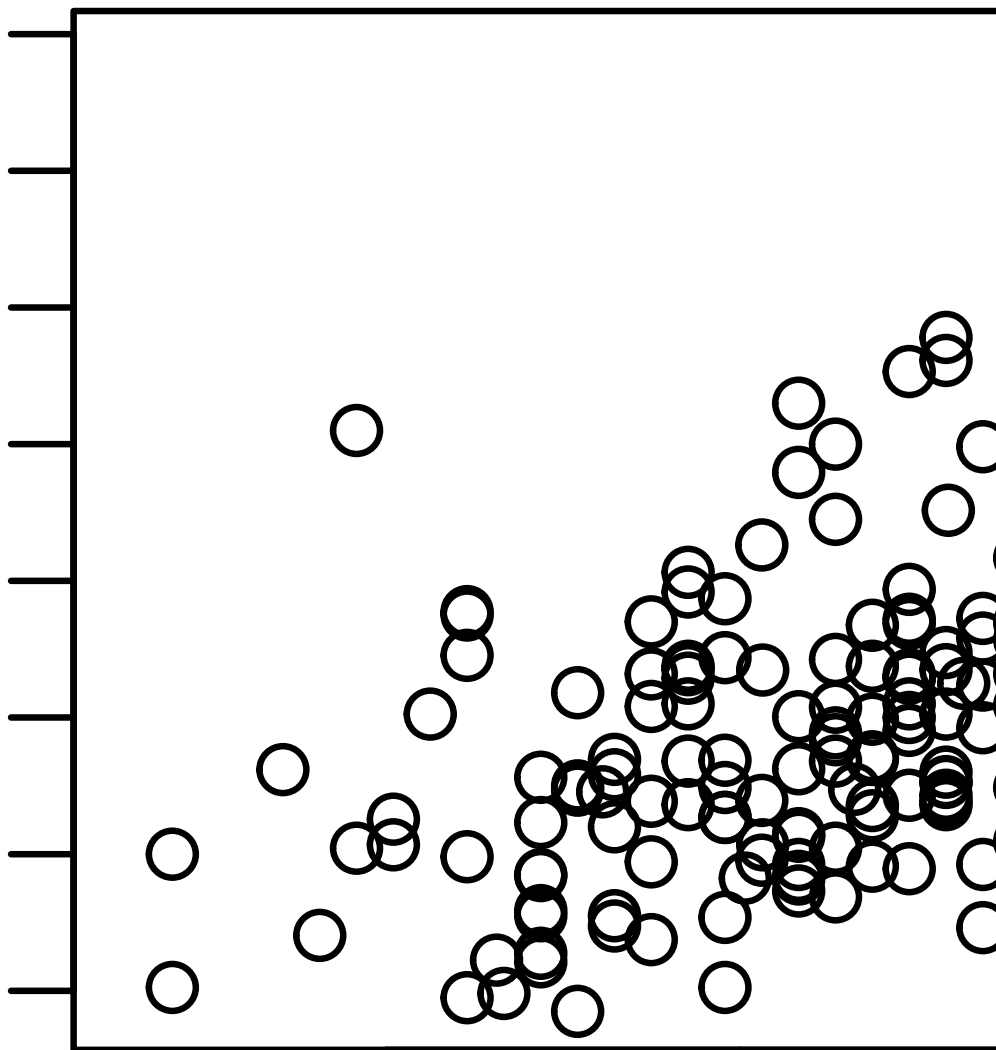
```
plot(d2$weight ~ d2$height)
```

d2\$weight

70 90 110 130

4.6

4.8



exercise

Update your mathematical model to incorporate height.

$$\begin{aligned}w_i &\sim \text{Normal}(\mu_i, \sigma) \\ \mu_i &= \alpha + \beta h_i \\ \alpha &\sim \text{Normal}(?, ?) \\ \beta &\sim \text{Normal}(0, 25) \\ \sigma &\sim \text{Uniform}(0, 25)\end{aligned}$$

= is deterministic – once we know other variables, μ_i is known with certainty
made-up parameters are the targets of learning

exercise

Update your mathematical model to incorporate height.

$$\begin{aligned}w_i &\sim \text{Normal}(\mu_i, \sigma) \\ \mu_i &= \alpha + \beta(h_i - \bar{h}) \\ \alpha &\sim \text{Normal}(130, 20) \\ \beta &\sim \text{Normal}(0, 25) \\ \sigma &\sim \text{Uniform}(0, 25)\end{aligned}$$

= is deterministic – once we know other variables, μ_i is known with certainty
made-up parameters are the targets of learning

$$\begin{aligned}
w_i &\sim \text{Normal}(\mu_i, \sigma) \\
\mu_i &= \alpha + \beta(h_i - \bar{h}) \\
\alpha &\sim \text{Normal}(130, 20) \\
\beta &\sim \text{Normal}(0, 25) \\
\sigma &\sim \text{Uniform}(0, 25)
\end{aligned}$$

To update our brms code:

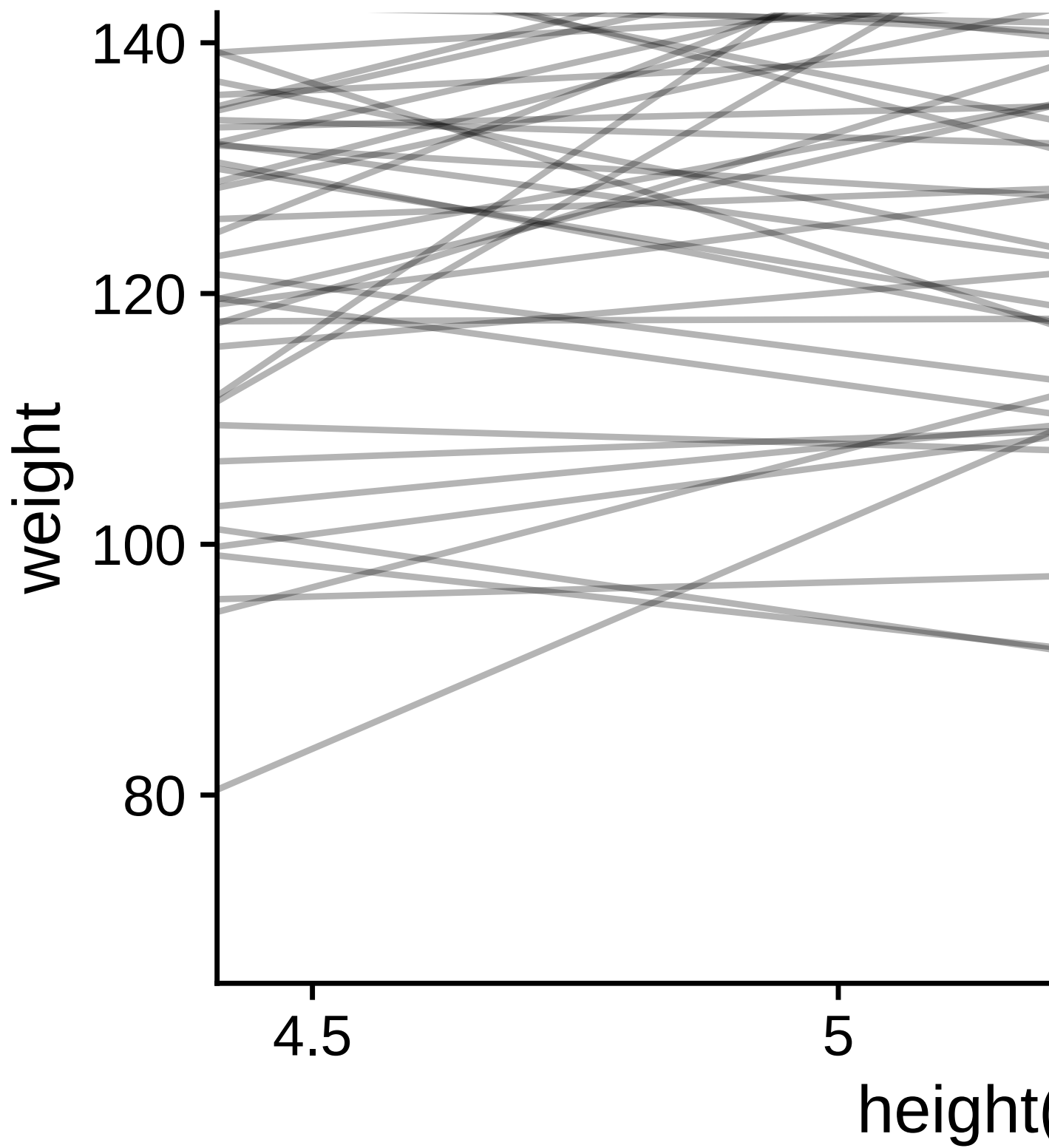
```
d2$height_c = d2$height - mean(d2$height)

m4p = brm(
  data = d2,
  family = gaussian,
  weight ~ 1 + height_c,
  prior = c(prior( normal(130,20), class=Intercept),
            prior( normal(0,25),   class=b),
            prior( uniform(0,25),  class=sigma, lb=0, ub=25)),
  iter = 5000, warmup = 1000, seed = 3, chains=1,
  sample_prior = "only")
```

```
set.seed(9)
samples_from_prior = as_draws_df(m4p)
str(samples_from_prior)
```

```
draws_df [4,000 x 9] (S3: draws_df/draws/tbl_df/tbl/data.frame)
 $ b_Intercept: num [1:4000] 126 142 148 118 115 ...
 $ b_height_c : num [1:4000] -16.652 -9.498 -12.363 0.248 -10.576 ...
 $ sigma      : num [1:4000] 2.63 19.37 16.55 4.3 12.92 ...
 $ Intercept  : num [1:4000] 126 142 148 118 115 ...
 $ lprior     : num [1:4000] -11.5 -11.5 -11.8 -11.5 -11.7 ...
 $ lp__       : num [1:4000] -10.66 -10.04 -10.08 -10.18 -9.83 ...
 $ .chain     : int [1:4000] 1 1 1 1 1 1 1 1 1 1 ...
 $ .iteration  : int [1:4000] 1 2 3 4 5 6 7 8 9 10 ...
 $ .draw      : int [1:4000] 1 2 3 4 5 6 7 8 9 10 ...
```

```
d2 %>%
  ggplot(aes(x=height_c, y=weight)) +
  geom_blank() +
  geom_abline( aes(intercept=b_Intercept, slope=b_height_c),
               data=samples_from_prior[1:50, ],
               alpha=.3) +
  scale_x_continuous(name = "height(feet)",
                     breaks=seq(4,6,by=.5)-mean(d2$height),
                     labels=seq(4,6,by=.5))
```

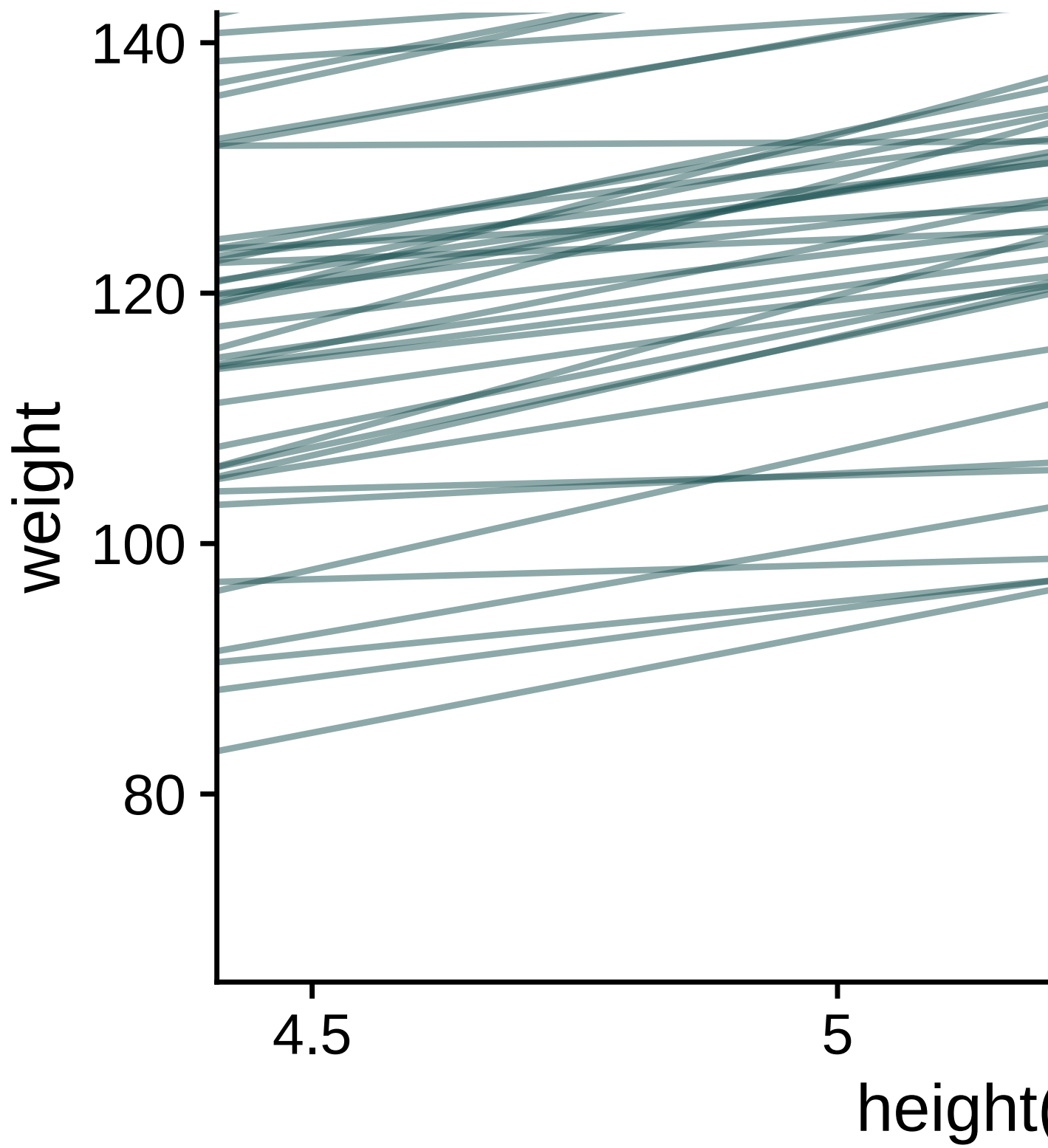


Describe in words what's wrong with our priors.

Slope should not be negative. How can we fix this?

Could use a uniform distribution bounded by 0.

```
m4p = brm(  
  data = d2,  
  family = gaussian,  
  weight ~ 1 + height_c,  
  prior = c(prior( normal(130,20), class=Intercept),  
            prior( uniform(0,25),   class=b),  
            prior( uniform(0,25),   class=sigma, lb=0, ub=25)),  
  iter = 5000, warmup = 1000, seed = 3, chains=1,  
  sample_prior = "only")
```



exercise

Fit the new weight model to the data.

solution

```
m4 = brm(  
  data = d2,  
  family = gaussian,  
  weight ~ 1 + height_c,  
  prior = c(prior( normal(130,20), class=Intercept),  
            prior( uniform(0,25), class=b),  
            prior( uniform(0,25), class=sigma, lb=0, ub=25)),  
  iter = 5000, warmup = 1000, seed = 3, chains=1,  
  file=here("files/models/m21.4"))  
posterior_summary(m4)
```

	Estimate	Est.Error	Q2.5	Q97.5
b_Intercept	99.21508	0.5154139	98.17317	100.20955
b_height_c	24.74661	0.2609169	24.05150	24.99355
sigma	10.36090	0.3898531	9.65234	11.12228
Intercept	99.21508	0.5154139	98.17317	100.20955
lprior	-11.53739	0.0396881	-11.61861	-11.46176
lp__	-1332.15882	1.3936194	-1335.90639	-1330.52002

exercise

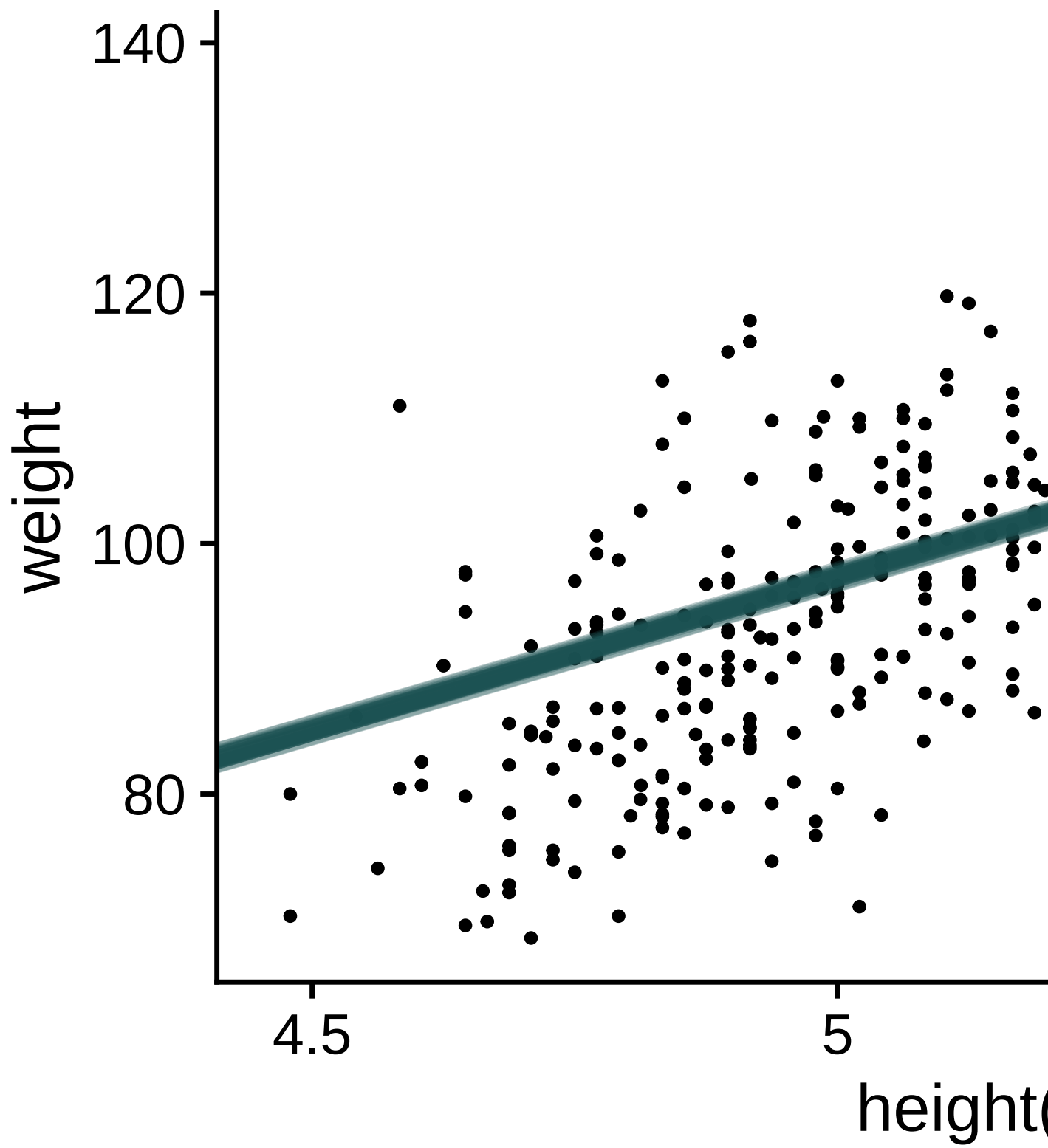
Draw lines from the posterior distribution and plot with the data.

solution

```

set.seed(9)
samples_from_posterior = as_draws_df(m4)
d2%>%
  ggplot(aes(x=height_c, y=weight)) +
  geom_point(size=.5) +
  geom_abline( aes(intercept=b_Intercept, slope=b_height_c),
               data=samples_from_posterior[1:50, ],
               alpha=.3,
               color="#1c5253") +
  scale_x_continuous(name = "height(feet)",
                     breaks=seq(4,6,by=.5)-mean(d2$height),
                     labels=seq(4,6,by=.5))

```



A side note: a major concern or critique of Bayesian analysis is that the subjectivity of the priors allow for nefarious behavior. “Putting our thumbs on the scale,” so to speak. But priors are quickly overwhelmed by data. Case in point:

```
1 m4e = brm(  
2   data = d2,  
3   family = gaussian,  
4   weight ~ 1 + height_c,  
5   prior = c(prior( normal(130,20), class=Intercept),  
6             prior( normal(-5,5),   class=b),  
7             prior( uniform(0,25),  class=sigma, lb=0, ub=25)),  
8   iter = 5000, warmup = 1000, seed = 3, chains=1,  
9   file=here("files/models/m21.4e"))  
10 posterior_summary(m4e)
```

	Estimate	Est.Error	Q2.5	Q97.5
b_Intercept	99.197733	0.5035653	98.214092	100.15932
b_height_c	35.775818	1.8832462	32.177572	39.50070
sigma	9.529429	0.3687178	8.839379	10.27233
Intercept	99.197733	0.5035653	98.214092	100.15932
lprior	-44.172476	3.0738968	-50.456409	-38.49994
lp__	-1334.629214	1.1849503	-1337.645046	-1333.23509

You’ll only really get into trouble with uniform priors that have a boundary, if true population parameter is outside your boundary. A good rule of thumb is to avoid the uniform distribution. We’ll cover other options for priors for σ in future lectures, but as a preview, the exponential distribution works very well for this!